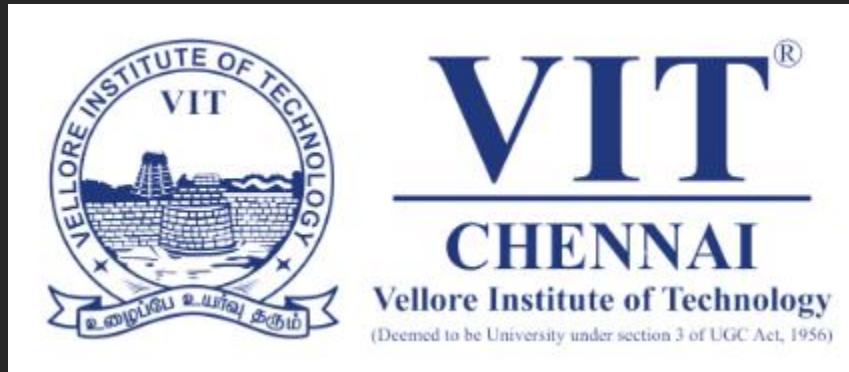


**VELLORE INSTITUTE OF TECHNOLOGY
CHENNAI CAMPUS**



**DATA SCIENCE PROJECT
WINTER SEMESTER 2025-26**

REVIEW – I

MEMBERS AND THEIR REGISTRATION NUMBERS.

- 1. RISHAV KUMAR – 24BCE1666**
- 2. JAYADITYA DUTTA – 24BCE1586**

FACULTY – DR. SUDHA C

COURSE CODE – BCSE206L

SLOT – B2 + TB2

**TITLE OF THE PROJECT – “From Data to Success:
Code-forces Analyzer for Performance Enhancement”**

Segment -> I - Project Idea

The project aims to predict the exam marks of students in a batch of 40 based on multiple academic and behavioral factors. These include:

- Current performance and previous academic records
- Attendance consistency
- Quality of notes and study materials
- Assignment & project completion quality
- Class participation
- Exam-taking skills (time management, anxiety, strategy)
- Cognitive and learning factors
- Study environment and psychological factors

The final output is to provide insights and personalized recommendations that help each student excel in their career and achieve their best potential.

Segment -> II - Work-frame / Approach

We will use Data Science concepts to analyze and predict student marks. The approach includes:

- Data Collection & Preprocessing: Gather student-related data, clean it, and normalize features.
- Feature Selection: Identify the most impactful factors (e.g., attendance, assignments, psychological scores).
- Modeling -
 - Apply Random Forest for feature importance and classification.
 - Use Neural Networks for prediction of marks with non-linear relationships.
- Visualization: Generate graphs to understand data behavior, such as:
 - Feature importance plots (Random Forest)
 - Accuracy/loss curves (Neural Network training)
 - Scatter plots comparing predicted vs actual marks
 - Confusion matrix for classification tasks

Segment -> Abstract

Competitive programming platforms generate large amounts of user performance data. Analyzing this data helps understand problem-solving skills, strengths, and weaknesses of programmers.

This project, Code-Forces Profile Analyzer, is a data science application that collects real-time data from the Code-Forces platform using public APIs and analyzes user performance based on rating history, contest participation, number of problems solved, and problem tags.

The system also allows comparison of two users and provides visual insights using graphs and charts.

Segment -> Problem Statement

To design and implement a data science-based system that:

- Fetches Code-Forces user data
 - Analyzes Contest performance and problem-solving behaviour.
 - Categorizes solved problems based on tags
 - Compares two users visually
 - Presents insights using interactive dashboards
-

Section → Procedure of Execution -

I. Data Preparation: Encode categorical variables, scale numerical values, and split into training/testing sets.

II. Model Training:

- Train Random Forest to evaluate feature importance.
- Train Neural Network to predict marks.

III. Graph Generation:

- Plot feature importance (bar chart).
- Plot training vs validation accuracy/loss (line graph).
- Plot predicted vs actual marks (scatter plot).

IV. Interpretation: Explain each graph—why it is used, how it is constructed, and what insights it provides.

Segment -> Objectives -

- To collect real-world competitive programming data using APIs
 - To apply data science lifecycle stages
 - To analyze user performance using data structures
 - To visualize trends and patterns
 - To compare strengths and weaknesses of two Code-Forces users
-

Segment -> Technology Stack -

Category	Tools Used
Programming Language	Python
Environment	WSL (Linux)
Data Source	Code-Forces Public API
Libraries	requests, pandas, matplotlib
Visualization	Matplotlib
UI Framework	Stream-Lit
Editor	VS Code

Segment -> Dataset Description -

5.1 Data Source

- Code-Forces Public API
- Live, real-time data (no static dataset)

5.2 API Endpoints Used

<u>API</u>	<u>Purpose</u>
“User - Info”	Profile details
“User – Rating”	Contest rating history
“User – Status”	Problem submissions

Segment -> Dataset Characteristics -

Characteristic	Description
Data Type	Structured (JSON)
Nature	Real-time
Size	Dynamic (depends on user activity)
Attributes	Numerical + Categorical
Missing Values	Some problems have no rating
Duplication	Multiple submissions per problem

Segment -> Data Science Life Cycle -

7.1 Data Collection

- Data fetched using REST APIs
- JSON responses converted into Python objects

7.2 Data Preprocessing

- Removed duplicate problem submissions using sets
- Filtered only accepted (OK) submissions
- Extracted relevant fields (rating, tags, contest)

7.3 Data Cleaning

- Ignored missing problem ratings
- Removed redundant contest submissions
- Validated usernames

7.4 Exploratory Data Analysis (EDA)

- Rating trends over contests
- Problem-solving distribution
- Tag-based frequency analysis

7.5 Data Analysis

- Count of problems solved
- Contest participation analysis
- Topic-wise strength using tags

7.6 Data Visualization

- Line graph for rating progress
- Bar charts for tag-wise problem distribution
- Comparison charts for two users

7.7 Interpretation

- Identify strong and weak problem areas
 - Compare competitive growth
 - Understand specialization topics
-

Segment -> GANTT CHART (PROJECT SCHEDULE)

<u>Task</u>	<u>Week 1</u>	<u>Week 2</u>	<u>Week 3</u>
Requirement Analysis	✓		
API Research	✓		
Data Collection		✓	
Data Cleaning		✓	
Visualization			✓
UI Development			✓
Report Writing			✓

Note -> (✓ indicates task duration)

Segment -> System Architecture -

User Input



Stream-Lit UI



Code-Forces API



Data Preprocessing



Data Analysis



Visualization & Comparison

Segment -> Applied Data Structures -

Structure

Purpose

List

Store ratings

Set

Avoid duplicate problems

Dictionary

Count tags

Data-Frame

Tabular comparison

Segment -> Results & Outputs-

11.1 Profile Summary

- Current rating
- Maximum rating
- Rank
- Contest count
- Total solved problems

11.2 Rating Analysis

- Rating growth trends
- Performance consistency

11.3 Tag-Based Analysis

- Top 10 problem tags
- Topic-wise strength

11.4 User Comparison

- Side-by-side comparison
 - Visual rating progress
-
-

Segment -> Data Interpretation

- Users with steady rating growth show consistent contest participation
 - Higher tag frequency indicates topic specialization
 - Tag analysis reveals strengths (e.g., DP, Greedy)
 - Sudden rating drops indicate contest inconsistency
 - Comparison helps identify skill gaps between users
-

Segment -> Future Scope

- Rating prediction using ML
 - Export reports as PDF
 - Leaderboard comparison
 - Topic recommendation system
 - Deployment on cloud
-

Segment -> Conclusion

The Code-Forces Profile Analyzer successfully demonstrates the application of data science concepts on real-world competitive programming data. The project integrates data collection, preprocessing, analysis, visualization, and interpretation in a complete pipeline. It effectively helps users understand their performance trends and compare skills, making it a valuable analytical tool.

Segment -> References

- [Code-Forces API Documentation](#)
- [Python Official Documentation](#)
- [Stream - Lit Documentation](#)

Segment -> Code

```
import streamlit as st

import requests

import pandas as pd

import matplotlib.pyplot as plt

from collections import defaultdict


st.set_page_config(page_title="Codeforces Profile Analyzer",
                    layout="wide")


# -----
# API Functions
# -----

def get_user_info(username):
    return requests.get(
        f"https://codeforces.com/api/user.info?handles={username}"
    ).json()


def get_rating_history(username):
    return requests.get(
        f"https://codeforces.com/api/user.rating?handle={username}"
    ).json()["result"]
```

```
def get_submissions(username):
    return requests.get(
        f"https://codeforces.com/api/user.status?handle={username}"
    ).json()["result"]

# -----
# Analyze Submissions
# -----

def analyze_submissions(submissions):
    solved = set()
    tag_count = defaultdict(int)

    for sub in submissions:
        if sub["verdict"] == "OK":
            prob = sub["problem"]
            pid = (prob["contestId"], prob["index"])

            if pid not in solved:
                solved.add(pid)
                for tag in prob["tags"]:
                    tag_count[tag] += 1
```

```

    return len(solved), dict(tag_count)

# -----
# UI
# -----

st.title("📊 Codeforces Profile Analyzer")

st.write("Compare two Codeforces users with tag-based problem analysis")

c1, c2 = st.columns(2)

user1 = c1.text_input("First Username")
user2 = c2.text_input("Second Username")

if st.button("Compare Profiles"):
    if not user1 or not user2:
        st.warning("Please enter both usernames")
    else:
        with st.spinner("Fetching data..."):
            u1_info = get_user_info(user1)
            u2_info = get_user_info(user2)

            if u1_info["status"] != "OK" or u2_info["status"] != "OK":

```

```

        st.error("Invalid username(s)")
    else:
        u1 = u1_info["result"][0]
        u2 = u2_info["result"][0]

        r1 = get_rating_history(user1)
        r2 = get_rating_history(user2)

        s1 = get_submissions(user1)
        s2 = get_submissions(user2)

        solved1, tags1 = analyze_submissions(s1)
        solved2, tags2 = analyze_submissions(s2)

        # -----
        # Summary Table
        # -----

        st.subheader("🔗 Profile Comparison")

        df = pd.DataFrame({
            user1: [
                u1.get("rating", "Unrated"),

```

```

        u1.get("maxRating", "N/A"),
        u1.get("rank", "N/A"),
        len(r1),
        solved1
    ],
    User2: [
        u2.get("rating", "Unrated"),
        u2.get("maxRating", "N/A"),
        u2.get("rank", "N/A"),
        len(r2),
        solved2
    ]
}, Index=[
    "Current Rating",
    "Max Rating",
    "Rank",
    "Contests",
    "Problems Solved"
])

st.table(df)

```

```

# -----

# Rating Graph

# -----

st.subheader("📈 Rating Progress Comparison")

fig, ax = plt.subplots()

ax.plot([x["newRating"] for x in r1], label=user1)
ax.plot([x["newRating"] for x in r2], label=user2)

ax.set_xlabel("Contest Number")
ax.set_ylabel("Rating")
ax.legend()
ax.grid()

st.pyplot(fig)

# -----

# TAG ANALYSIS

# -----

st.subheader("🔖 Problems Solved by Tags")

col1, col2 = st.columns(2)

```

```
# User 1 Tags

top_tags1 = dict(sorted(tags1.items(), key=lambda x: x[1],
reverse=True)[:10])

col1.write(f'**Top Tags – {user1}**')


fig1, ax1 = plt.subplots()
ax1.bar(top_tags1.keys(), top_tags1.values())
ax1.set_xticklabels(top_tags1.keys(), rotation=45, ha="right")
ax1.set_ylabel("Problems Solved")
col1.pyplot(fig1)


# User 2 Tags

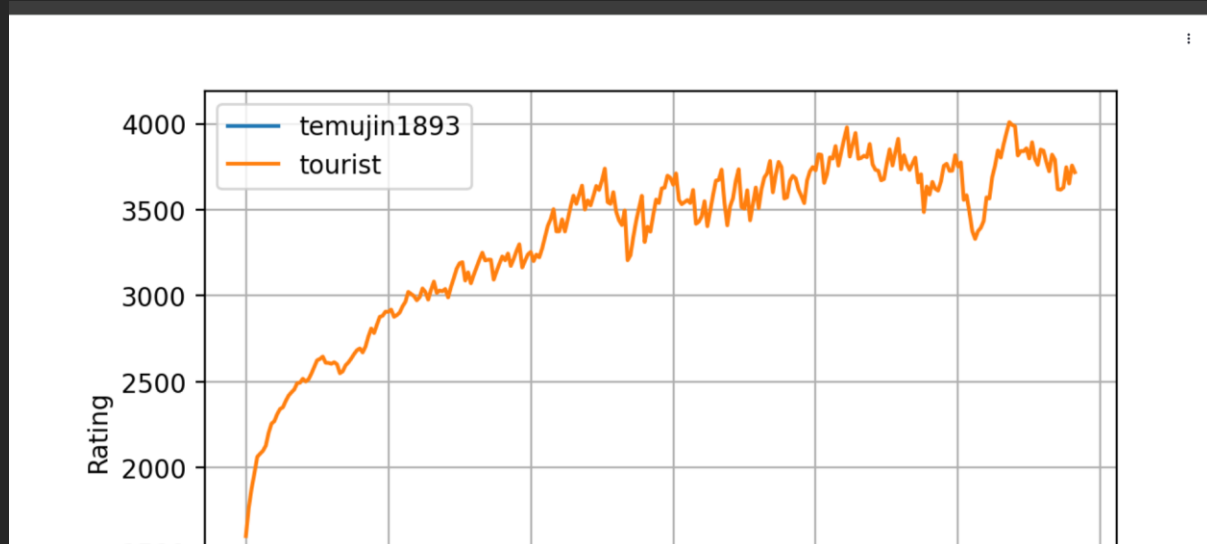
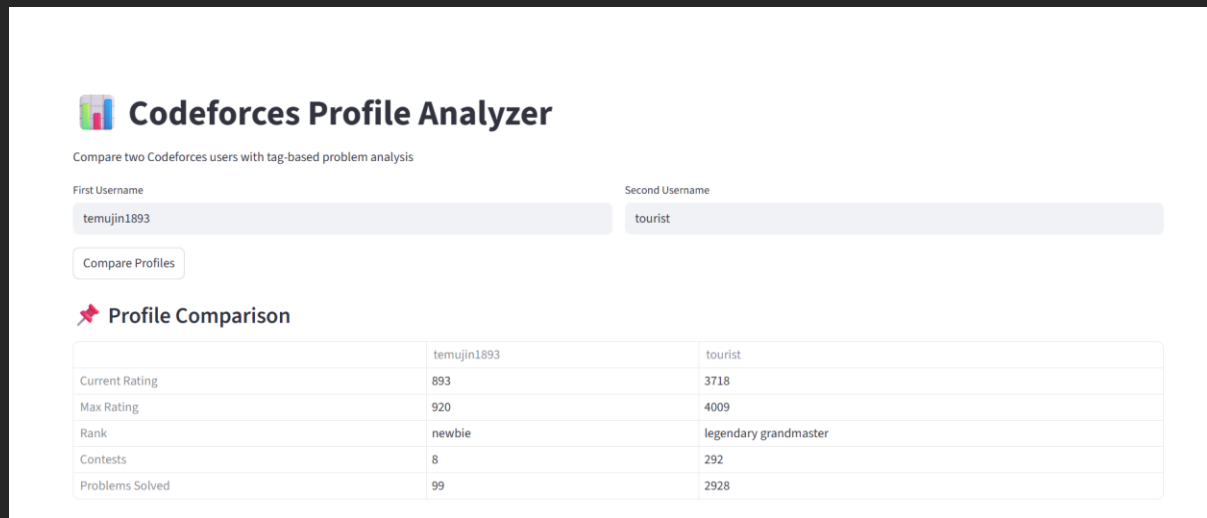
top_tags2 = dict(sorted(tags2.items(), key=lambda x: x[1],
reverse=True)[:10])

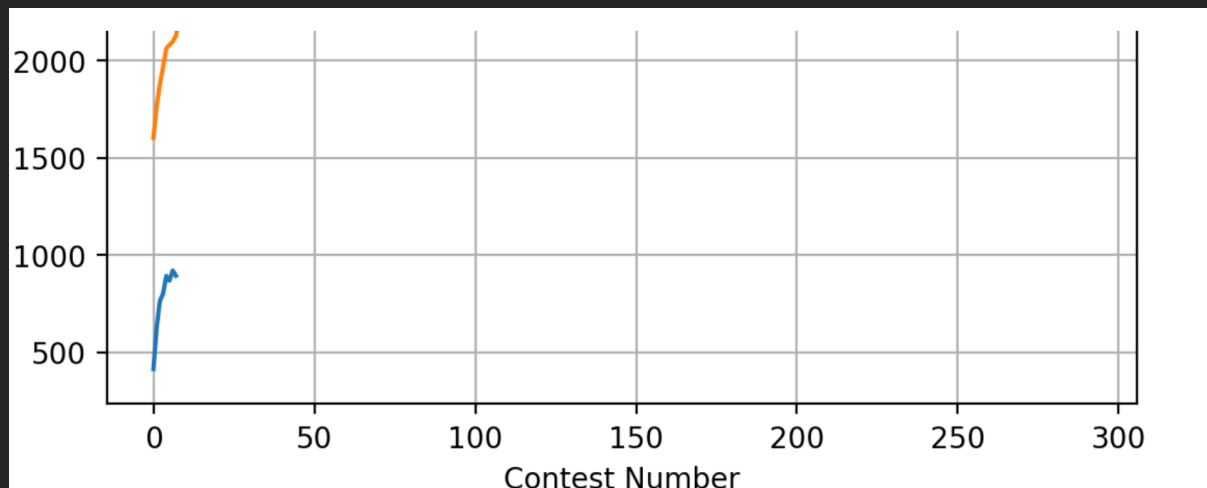
col2.write(f'**Top Tags – {user2}**')


fig2, ax2 = plt.subplots()
ax2.bar(top_tags2.keys(), top_tags2.values())
ax2.set_xticklabels(top_tags2.keys(), rotation=45, ha="right")
ax2.set_ylabel("Problems Solved")
col2.pyplot(fig2)
```

st.success("Tag-based comparison completed 🍷")

Segment -> Screenshots of the Results.





Segment -> Data Analysis Graphs.

